REVISÃO TÉCNICA · ENADE · COMPUTAÇÃO

Orientação a Objetos

Polimorfismo, Herança, UML, Padrões de Projeto, Abstração e Encapsulamento — 6 questões comentadas em profundidade

Q1 Polimorfismo | Q2 Herança | Q3 UML | Q4 GoF · Strategy | Q5 Abstração | Q6 Encapsulamento

AGENDA DA AULA

Seis questões integradoras de Orientação a Objetos

Nota de alinhamento: as 6 questões integram os 8 tópicos da Matriz de Referência; alguns tópicos são avaliados conjuntamente.

Q1

Polimorfismo e Vinculação Dinâmica

Vinculação estática x dinâmica, Template Method e field shadowing

Q2

Herança (é-um) x Composição (tem-um)

Liskov, acoplamento e o uso correto de interfaces

Q3

Diagramas de Classes UML e Multiplicidade

Associação, agregação, composição e cardinalidades

Q4

Padrões de Projeto GoF — Strategy

Princípio Aberto/Fechado e eliminação de condicionais

Q5

Classes Abstratas x Interfaces

Compartilhamento de estado x contratos comportamentais

Q6

Encapsulamento

Tell, Don't Ask e o Modelo de Domínio Anêmico

Q1

QUESTÃO 1 · QUESTÃO AUTORAL NO ESTILO ENADE

Enunciado da Questão

CONTEXTUALIZAÇÃO

A implementação de regras de negócio em sistemas orientados a objetos frequentemente se vale do padrão comportamental Template Method, que define o esqueleto de um algoritmo na superclasse e delega etapas específicas para as subclasses. A forma como os atributos e métodos interagem nesse cenário exige compreensão rigorosa sobre escopo, encapsulamento e vinculação (binding).

COMANDO

Considerando os princípios de polimorfismo, vinculação dinâmica (dynamic binding) e boas práticas de encapsulamento, avalie o comportamento do código executado na classe SistemaPrincipal e a qualidade arquitetural da decisão de implementação adotada na relação de herança. Assinale a alternativa tecnicamente adequada:

Q1

QUESTÃO 1 · CENÁRIO

Qual o valor impresso por p.processar(100.0)?

java

```java
public abstract class ProcessadorDeRegra {
    protected double fatorMultiplicador = 1.5;

    public double processar(double valorBase) {
        return (valorBase * fatorMultiplicador) + calcularTaxaExtra();
    }

    protected double calcularTaxaExtra() {
        return 10.0;
    }
}

public class ProcessadorComplexo extends ProcessadorDeRegra {
    protected double fatorMultiplicador = 2.0;

    @Override
    protected double calcularTaxaExtra() {
        return 25.0;
    }
}

public class SistemaPrincipal {
    public static void main(String[] args) {
        ProcessadorDeRegra p = new ProcessadorComplexo();
        System.out.println(p.processar(100.0));
    }
}
```

Q1

QUESTÃO 1 · ALTERNATIVAS

Qual alternativa é tecnicamente adequada?

A A saída será 225.0, pois a vinculação dinâmica seria aplicada tanto ao método calcularTaxaExtra() (25.0) quanto ao atributo fatorMultiplicador (2.0) da subclasse. O projeto teria alta coesão, pois delega às subclasses a gestão de seus próprios estados e comportamentos.

B A saída será 160.0, pois a tipagem da referência p forçaria vinculação estática para TODOS os membros e métodos de ProcessadorDeRegra. O projeto seria excelente em segurança e manutenibilidade, blindando o método principal contra alterações das subclasses.

C A saída será 225.0, pois a subclasse redefiniria a alocação de memória do atributo fatorMultiplicador para 2.0. O projeto aumentaria indevidamente o acoplamento, pois obrigaria a classe base a conhecer e invocar estados modificados pelas subclasses.

D A saída será 175.0, pois fatorMultiplicador (1.5) sofre vinculação estática pela classe base, enquanto calcularTaxaExtra() (25.0) sofre vinculação dinâmica. O projeto é inadequado: o shadowing quebra o encapsulamento; o estado deveria ser configurado via construtores, não recriado nas subclasses.

Q1

QUESTÃO 1 · GABARITO COMENTADO

Resposta Correta: Alternativa D

✓ **Por que a alternativa D está correta**

O método processar() está definido na SUPERCLASSE, então a leitura de fatorMultiplicador ocorre no contexto de ProcessadorDeRegra → valor 1.5 (vinculação estática de atributo). Já a chamada calcularTaxaExtra() sofre vinculação dinâmica: como o objeto real é ProcessadorComplexo, executa a versão sobrescrita → 25.0. Cálculo: (100 × 1.5) + 25 = 175.0. No plano arquitetural, o field shadowing é uma falha de projeto: cria dois estados "fatorMultiplicador" distintos na memória do mesmo objeto. A correção adequada é inicializar o valor herdado via construtor (super(2.0)) ou expor um método polimórfico getFator().

POR QUE AS DEMAIS ALTERNATIVAS ERRAM

A Assume, erroneamente, que atributos são polimórficos como os métodos — leva ao cálculo incorreto (100×2.0)+25=225.0 e classifica o shadowing como boa prática de coesão.

B Ignora a sobrescrita (@Override) de calcularTaxaExtra() e assume vinculação estática também para métodos — resulta em (100×1.5)+10=160.0 e trata a ausência de polimorfismo como vantagem de segurança.

C Chega ao mesmo valor incorreto da alternativa A (225.0), mas erra ao falar em "realocação de memória do atributo" — não há redefinição de alocação; apenas um novo campo oculta o da superclasse.

Q1

QUESTÃO 1 · CONCEITO CONSOLIDADO

Polimorfismo e Vinculação Dinâmica

- **Vinculação estática (early binding):** resolvida em tempo de COMPILAÇÃO. Em Java, TODOS os atributos (campos) usam esse mecanismo — o tipo DECLARADO da referência decide qual campo é acessado.

- **Vinculação dinâmica (late binding):** resolvida em tempo de EXECUÇÃO. Aplica-se a métodos de instância sobrescritos com @Override — o tipo REAL do objeto decide qual versão do método roda.

- **Template Method (padrão comportamental do GoF):** a superclasse define o esqueleto do algoritmo (processar()) e delega etapas variáveis às subclasses (calcularTaxaExtra()).

- **Field shadowing (ocultação de campos):** quando a subclasse declara um atributo com o mesmo nome da superclasse, cria-se um SEGUNDO campo na memória — prática fortemente desaconselhada, pois gera ambiguidade de estado.

PRINCÍPIO-CHAVE Atributo → vinculação estática (classe declarada) | Método sobrescrito → vinculação dinâmica (classe real do objeto)

Q2

QUESTÃO 2 · QUESTÃO AUTORAL NO ESTILO ENADE

Enunciado da Questão

CONTEXTUALIZAÇÃO

Um arquiteto de software está revisando o código desenvolvido por um programador júnior para o módulo de controle de um sistema de Internet das Coisas (IoT) voltado para automação industrial. O sistema precisa lidar com diversos sensores que coletam dados e, quando necessário, emitem alertas de segurança enviando dados através da rede. O programador propôs a estrutura apresentada a seguir para resolver a necessidade de comunicação de rede e o comportamento de atuação do sensor de temperatura.

COMANDO

Avaliando a estrutura de herança e implementação apresentada no código, e considerando os impactos dessa decisão sobre o acoplamento, coesão, extensibilidade e a semântica estrutural da orientação a objetos, qual alternativa apresenta a análise técnica mais adequada?

Q2

QUESTÃO 2 · CENÁRIO

A herança de ComunicadorRede é uma boa decisão de projeto?

java

```java
public class ComunicadorRede {
    public void estabelecerConexao(String ip) {
        // Lógica de infraestrutura para handshake e conexão TCP/IP
    }
    public void enviarPacote(byte[] dados) {
        // Lógica genérica de roteamento e envio de pacotes na rede
    }
}

public interface DispositivoAtuador {
    void acionarAlerta(String mensagem);
}

// Modelagem proposta pelo desenvolvedor
public class SensorTemperaturaSmart extends ComunicadorRede
        implements DispositivoAtuador {
    private double temperaturaAtual;

    public void medirTemperatura() {
        // Lógica de hardware que atualiza temperaturaAtual
    }
    @Override
    public void acionarAlerta(String mensagem) {
        if (this.temperaturaAtual > 40.0) {
            enviarPacote(mensagem.getBytes()); // reaproveita método herdado
        }
    }
}
```

Q2

QUESTÃO 2 · ALTERNATIVAS

Qual alternativa é tecnicamente adequada?

A A implementação da interface está incorreta, pois interfaces em Java serviriam para simular herança múltipla de ESTADO; a ação de alerta deveria migrar para uma superclasse abstrata para herdar atributos.

B A herança simples atende ao princípio de especialização, tornando o sensor um subtipo de comunicador. A falha estaria no uso da interface, que impediria o reaproveitamento do código do método de acionamento.

C A herança de ComunicadorRede é inadequada por não representar uma relação semântica "é-um" adequada, expondo métodos de infraestrutura ao domínio do sensor e podendo quebrar a substituibilidade esperada. O reaproveitamento de código de rede deveria ocorrer por composição.

D A modelagem é correta e eficiente, pois a herança garante reaproveitamento nativo do envio de dados, evitando duplicação de código e compensando a falta de herança múltipla na linguagem.

Q2

QUESTÃO 2 · GABARITO COMENTADO

Resposta Correta: Alternativa C

✓ **Por que a alternativa C está correta**

SensorTemperaturaSmart não é semanticamente um ComunicadorRede — ele apenas UTILIZA capacidades de rede para enviar alertas. Herdar somente para reaproveitar enviarPacote() pode violar a substituibilidade e expõe indevidamente métodos como estabelecerConexao() a qualquer cliente do sensor. A correção é aplicar composição: o sensor deveria possuir um atributo privado do tipo ComunicadorRede. Já o uso da interface DispositivoAtuador está correto — define um contrato de comportamento (polimorfismo) sem acoplar implementação.

POR QUE AS DEMAIS ALTERNATIVAS ERRAM

A Erro conceitual grave: interfaces NÃO armazenam estado e não servem para "simular herança múltipla de estado". O uso da interface no cenário já está correto.

B Aceita erroneamente a premissa de "especialização" entre sensor e comunicador, e culpa a interface por não reaproveitar código — papel que nunca foi dela.

D Acredita que o propósito da herança é evitar duplicação de código a qualquer custo — ignora que o acoplamento rígido resultante engessa a evolução do sistema (ex.: adicionar Bluetooth quebraria a herança).

Q2

QUESTÃO 2 · CONCEITO CONSOLIDADO

Herança (é-um) x Composição (tem-um)

- **Princípio da Substituição de Liskov (LSP):** uma subclasse deve poder substituir sua superclasse sem quebrar o comportamento esperado — herança só é apropriada para uma relação "é-um" genuína.

- **"Prefira composição à herança" (Effective Java, item 18 / GoF):** quando o objetivo é apenas reaproveitar funcionalidades de outra classe, o caminho correto é TER uma referência a ela (composição), não estendê-la.

- **Interfaces:** definem CONTRATOS de comportamento e habilitam polimorfismo. Não possuem estado mutável de instância, mas podem declarar constantes e métodos default compartilhados.

- **Riscos da herança indevida:** acoplamento rígido, exposição de métodos internos da superclasse a qualquer cliente, e dificuldade de evoluir o sistema (ex.: trocar o meio de comunicação de rede).

PRINCÍPIO-CHAVE Herança modela "é-um"; composição modela "tem-um" — nunca herde apenas para reaproveitar código

Q3

QUESTÃO 3 · QUESTÃO AUTORAL NO ESTILO ENADE

Enunciado da Questão

CONTEXTUALIZAÇÃO

A correta tradução de regras de negócio para a estrutura estática de um software é uma etapa crítica na engenharia de requisitos e no projeto orientado a objetos, garantindo que o sistema imponha as restrições de integridade do domínio antes mesmo da implementação do código-fonte.

COMANDO

Considerando a correspondência rigorosa entre o modelo UML e as regras de negócio apresentadas, analise os relacionamentos e avalie qual alternativa traduz corretamente as associações, papéis e multiplicidades para o Diagrama de Classes:

Q3

QUESTÃO 3 · CENÁRIO

Traduzindo regras de negócio em relacionamentos UML

Veterinário ↔ Atendimento

Um Veterinário pode realizar diversos Atendimentos ao longo do mês, mas também pode não ter realizado nenhum. Cada Atendimento é de responsabilidade de exatamente um Veterinário.

Tutor ↔ Animal

Um Animal deve possuir um e apenas um Tutor legal vinculado. Para existir no sistema, um Tutor precisa ter sob sua responsabilidade um ou mais Animais.

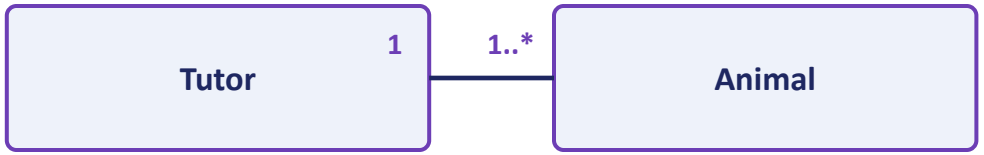
Atendimento ↔ Receituário

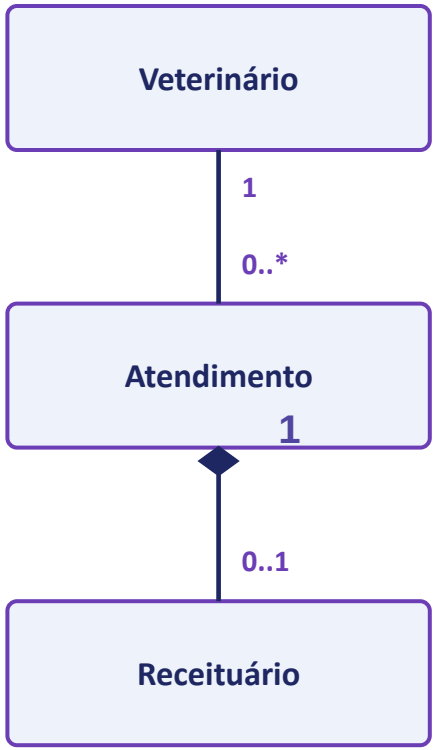
Ao final do Atendimento, o sistema pode gerar no máximo um Receituário. Um Receituário tem pertencimento estrutural e ciclo de vida associado ao Atendimento que o gerou.

Q3

QUESTÃO 3 · MODELAGEM UML

Diagrama de Classes Correto





◆ Losango preenchido = composição: o Receituário tem ciclo de vida associado ao Atendimento

Leitura explícita: cada Animal → 1 Tutor; cada Tutor → 1..* Animais; cada Atendimento → 1 Veterinário; cada Veterinário → 0..* Atendimentos; cada Atendimento → 0..1 Receituário

Q3

QUESTÃO 3 · ALTERNATIVAS

Qual alternativa traduz corretamente as multiplicidades?

A Tutor-Animal: [1..*] em ambos os lados (muitos-para-muitos). Veterinário-Atendimento: [0..*] no lado do Atendimento. Atendimento-Receituário: Herança, com Receituário especializando Atendimento.

B Tutor-Animal: [1] no Tutor e [1..*] no Animal. Veterinário-Atendimento: [1] no Veterinário e [0..*] no Atendimento. Atendimento-Receituário: Composição (losango no Atendimento [1]), multiplicidade [0..1] no Receituário.

C Tutor-Animal: [1..*] no Tutor e [1] no Animal. Veterinário-Atendimento: [0..*] no Veterinário e [1] no Atendimento. Atendimento-Receituário: Agregação (losango vazio no Receituário [0..1]).

D Tutor-Animal: [1] no Tutor e [0..*] no Animal. Veterinário-Atendimento: [1..*] apontando para Atendimento. Atendimento-Receituário: Associação simples, multiplicidade [1..*] em Receituário.

Q3

QUESTÃO 3 · GABARITO COMENTADO

Resposta Correta: Alternativa B

Por que a alternativa B está correta

Tutor (1) — Animal (1..*): um tutor tem um ou mais animais; cada animal tem exatamente um tutor. Veterinário (1) — Atendimento (0..*): um veterinário realiza zero ou muitos atendimentos; cada atendimento pertence a exatamente um veterinário. Atendimento–Receituário: Composição (losango no Atendimento), pois o receituário tem pertencimento estrutural e ciclo de vida associado ao atendimento; multiplicidade 0..1 no Receituário, já que é opcional e único.

POR QUE AS DEMAIS ALTERNATIVAS ERRAM

A Propõe multiplicidade muitos-para-muitos entre Tutor e Animal (ignora "exatamente um Tutor") e modela Receituário como Herança — um receituário não "é um" atendimento, ele é GERADO por um atendimento.

C Inverte a leitura da multiplicidade (coloca 1..* no Tutor, não no Animal) e confunde Composição com Agregação, posicionando o losango na classe PARTE (Receituário) em vez do TODO.

D Fere a regra de "um ou mais animais" ao usar 0..* em vez de 1..*, e transforma a composição em associação simples 1..* — tornando o receituário obrigatório e múltiplo (o oposto de "no máximo um").

Q3

QUESTÃO 3 · CONCEITO CONSOLIDADO

Diagramas de Classes UML — Associação, Agregação, Composição

- **Associação simples:** relação estrutural entre classes (linha); a multiplicidade em cada extremidade indica quantas instâncias participam da relação.

- **Agregação (losango vazio):** relação conjunto–parte fraca; a parte pode existir independentemente do conjunto.

- **Composição (losango preenchido):** relação conjunto–parte forte, com pertencimento estrutural e ciclo de vida fortemente associado ao conjunto. O losango fica no lado do conjunto.

- **Leitura da multiplicidade:** o número exibido próximo a uma classe indica quantas instâncias DAQUELA classe se relacionam com UMA instância do lado oposto (1, 0..1, 0..*, 1..*).

PRINCÍPIO-CHAVE Composição = losango preenchido no conjunto · Agregação = losango vazio · multiplicidade lê-se do lado oposto

Q4

QUESTÃO 4 · QUESTÃO AUTORAL NO ESTILO ENADE

Enunciado da Questão

CONTEXTUALIZAÇÃO

A manutenção de sistemas legados frequentemente revela problemas arquiteturais decorrentes do crescimento orgânico do software. Decisões de projeto inadequadas podem levar a altos níveis de acoplamento e violações de princípios basilares da orientação a objetos, dificultando a extensibilidade.

COMANDO

Considerando a necessidade de reduzir o acoplamento, aumentar a coesão e permitir a introdução de novas modalidades sem modificar o código existente da classe CalculadoraFrete, qual padrão de projeto e respectiva estratégia de implementação resolvem arquiteturalmente o problema apresentado?

Q4

QUESTÃO 4 · CENÁRIO

Como extensibilizar o cálculo de frete sem alterar a classe atual?

Contexto: a equipe precisa adicionar 4 novas modalidades de entrega (ex.: "Retirada na Loja", "Frete Internacional"). A cada nova regra, CalculadoraFrete precisa ser modificada — gerando conflitos de versão e risco de bugs em lógicas que já funcionavam.

java

```java
public class CalculadoraFrete {

    public double calcular(Pedido pedido, String modalidade) {
        if (modalidade.equals("PAC")) {
            // Lógica complexa de 15 linhas envolvendo dimensões e peso
            return pedido.getPeso() * 1.2;
        } else if (modalidade.equals("SEDEX")) {
            // Lógica complexa de 20 linhas com taxas de urgência
            return (pedido.getPeso() * 2.5) + 15.0;
        } else if (modalidade.equals("TRANSPORTADORA_X")) {
            // Lógica complexa de 30 linhas com pedágio e cubagem
            return (pedido.getVolume() * 1.5) + 50.0;
        }
        throw new IllegalArgumentException("Modalidade inválida");
    }
}
```

Q4

QUESTÃO 4 · ALTERNATIVAS

Qual padrão resolve arquiteturalmente o problema?

A Aplicar o padrão Strategy, encapsulando cada lógica de cálculo em classes concretas distintas que implementam uma interface comum. A calculadora recebe o comportamento por composição, eliminando condicionais e respeitando o OCP.

B Aplicar o padrão Factory Method, criando uma superclasse abstrata de calculadora que delega a instanciação dos cálculos às subclasses, mantendo as condicionais centralizadas em um método criacional unificado.

C Aplicar o padrão Observer, transformando a calculadora em um Subject que emite eventos para as transportadoras (Observers), descentralizando o cálculo e permitindo adicionar modais dinamicamente.

D Aplicar o padrão Singleton na classe de cálculo, garantindo estado global compartilhado entre módulos, evitando recriação de objetos e concentrando toda a variação algorítmica em uma instância única.

Q4

QUESTÃO 4 · GABARITO COMENTADO

Resposta Correta: Alternativa A

Por que a alternativa A está correta

O problema central é a variação de algoritmos (regras de cálculo de frete) concentrada em uma única classe, violando o Princípio Aberto/Fechado. O Strategy resolve criando uma interface comum (ex.: EstrategiaFrete) e classes concretas (CalculoPac, CalculoSedex...). A CalculadoraFrete passa a interagir apenas com a interface, delegando a execução do cálculo e retirando a lógica condicional do algoritmo principal, promovendo alta coesão (cada regra isolada em sua classe) e permitindo novas modalidades sem alterar o código base.

POR QUE AS DEMAIS ALTERNATIVAS ERRAM

B Confunde a finalidade do Factory Method (criacional — decide QUEM instancia o objeto) com o problema real, que exige encapsular COMPORTAMENTOS/algoritmos intercambiáveis; as condicionais permaneceriam.

C Aplica Observer, destinado à notificação de mudanças de estado entre múltiplos observadores (1-N), a um problema que exige um único algoritmo síncrono retornando um valor de cálculo.

D Propõe Singleton (criacional, garante instância única) para um problema comportamental — não elimina as condicionais nem melhora a coesão, e ainda introduz estado global como prática questionável.

Q4

QUESTÃO 4 · CONCEITO CONSOLIDADO

Padrão Strategy e o Princípio Aberto/Fechado (OCP)

- **O problema:** estruturas condicionais (if/else) que crescem a cada nova regra de negócio, aumentando o acoplamento e violando o OCP — o software deve estar aberto para EXTENSÃO e fechado para MODIFICAÇÃO.

- **Strategy (padrão comportamental do GoF):** encapsula cada algoritmo em uma classe própria, todas implementando uma interface comum. O contexto passa a receber o comportamento por composição, sem concentrar as condicionais no algoritmo principal.

- **Diferença rápida:** Strategy = algoritmos intercambiáveis (comportamental) · Factory Method = quem cria o objeto (criacional) · Observer = notificação um-para-muitos · Singleton = instância única (criacional).

- **Benefício direto:** retira a lógica condicional do contexto principal, aumenta a coesão (cada regra isolada em sua própria classe) e permite adicionar modalidades apenas criando novas classes, sem tocar no código existente.

PRINCÍPIO-CHAVE Se você está adicionando "else if" a cada nova regra de negócio, provavelmente precisa de um Strategy

Q5

QUESTÃO 5 · QUESTÃO AUTORAL NO ESTILO ENADE

Enunciado da Questão

CONTEXTUALIZAÇÃO

A arquitetura de sistemas orientados a objetos frequentemente exige a definição precisa de fronteiras contratuais e pontos de compartilhamento de código. A escolha inadequada entre mecanismos de abstração pode resultar em forte acoplamento, duplicação de lógica ou engessamento da evolução do software.

COMANDO

Considerando os requisitos de compartilhamento de estado, multiplicidade de comportamentos e a necessidade de classes de diferentes hierarquias cumprirem os mesmos contratos de forma desacoplada, avalie as alternativas de modelagem estrutural em Java e assinale a decisão de projeto tecnicamente mais adequada:

Q5

QUESTÃO 5 · CENÁRIO

Como modelar comportamentos que cruzam hierarquias diferentes?

```java
public abstract class DispositivoBase {
    protected String idRegistro;
    protected String enderecoRede;

    public void conectarBase() {
        // Lógica padronizada de handshake
        // e autenticação na rede
    }

    public abstract void iniciarOperacao();
}
```

Desafio: como compartilhar estado (idRegistro, enderecoRede) e, ao mesmo tempo, permitir múltiplos comportamentos cruzando hierarquias distintas?

NOVOS REQUISITOS

- TermostatoSmart: dispositivo padrão da rede, mas também precisa ser Controlavel (ligar/desligar remotamente) e Auditavel (registrar logs).
- CameraSeguranca: dispositivo da rede, deve ser Auditavel e TransmissorVideo.
- Auditavel também deve ser adotado por ServidorNuvem e GatewayDePagamento — módulos SEM nenhuma relação com dispositivos físicos.

Q5

QUESTÃO 5 · ALTERNATIVAS

Qual decisão de projeto é tecnicamente mais adequada?

A Converter todos os elementos, incluindo DispositivoBase, em interfaces, exigindo que cada dispositivo concreto implemente do zero sua lógica de conexão e declare seus próprios atributos de registro.

B Definir Controlavel e Auditavel como classes abstratas (especializações do domínio) e refatorar DispositivoBase como interface, aproveitando a suposta capacidade de interfaces armazenarem estado mutável.

C Manter DispositivoBase como classe abstrata (reúso de estado e conexão). Modelar Controlavel e Auditavel como interfaces, permitindo múltiplos contratos no TermostatoSmart e que ServidorNuvem assuma esses comportamentos sem acoplamento indevido.

D Transformar Controlavel e Auditavel em classes abstratas. Como Java não suporta herança múltipla, organizar em hierarquia vertical: TermostatoSmart herda de Controlavel, que herda de Auditavel, que deriva de DispositivoBase.

Q5

QUESTÃO 5 · GABARITO COMENTADO

Resposta Correta: Alternativa C

✓ **Por que a alternativa C está correta**

DispositivoBase permanece como classe abstrata, compartilhando estado (idRegistro, enderecoRede) e comportamento concreto (conectarBase()) entre os dispositivos IoT. Controlavel e Auditavel são modeladas como interfaces, pois representam capacidades que atravessam hierarquias distintas — TermostatoSmart implementa ambas, e classes sem relação com dispositivos físicos (ServidorNuvem, GatewayDePagamento) também podem ser Auditavel sem herdar características indesejadas de um dispositivo IoT.

POR QUE AS DEMAIS ALTERNATIVAS ERRAM

A Transforma tudo em interface, inclusive DispositivoBase — obrigaria a duplicar atributos e a lógica de conectarBase() em cada dispositivo concreto, violando o princípio DRY (Don't Repeat Yourself).

B Erro técnico grave: interfaces não podem armazenar estado de instância mutável em Java — tornar DispositivoBase interface inviabilizaria o armazenamento de idRegistro e enderecoRede.

D Cria uma hierarquia linear artificial para contornar a ausência de herança múltipla — fere a semântica da OO e impediria ServidorNuvem de ser Auditavel sem também se tornar um DispositivoBase.

Q5

QUESTÃO 5 · CONCEITO CONSOLIDADO

Classes Abstratas x Interfaces

- **Classe abstrata:** modela a hierarquia principal de uma relação "é-um"; permite compartilhar ESTADO (atributos) e implementação concreta entre subclasses. Em Java, só é possível herdar de UMA classe.

- **Interface:** modela uma capacidade transversal ("pode-fazer"), definindo um contrato de comportamento; uma classe pode implementar VÁRIAS interfaces, mesmo em hierarquias diferentes.

- **Restrição da linguagem:** interfaces NÃO armazenam estado de instância mutável — apenas constantes (public static final).

- **Regra de decisão:** hierarquia natural + estado compartilhado → classe abstrata. Comportamento que atravessa hierarquias não relacionadas → interface.

PRINCÍPIO-CHAVE Estado compartilhado + hierarquia natural → classe abstrata · Comportamento transversal → interface

Q6

QUESTÃO 6 · QUESTÃO AUTORAL NO ESTILO ENADE

Enunciado da Questão

CONTEXTUALIZAÇÃO

O encapsulamento é um princípio fundamental da programação orientada a objetos que vai além da simples aplicação de modificadores de visibilidade para ocultar atributos. Seu objetivo central é garantir que um objeto tenha controle exclusivo sobre seu estado interno, protegendo suas invariantes e concentrando os comportamentos que alteram esse estado. A aplicação inadequada desse princípio pode levar à criação de modelos de domínio anêmicos, nos quais os objetos atuam como meros repositórios de dados expostos, aumentando drasticamente o acoplamento estrutural e dificultando a manutenção do sistema.

COMANDO

Considerando os princípios de orientação a objetos, a preservação do estado interno das entidades e a necessidade de reduzir o acoplamento entre as classes, avalie a implementação apresentada e assinale a alternativa que descreve a decisão de projeto mais adequada para aplicar o encapsulamento neste cenário:

Q6

QUESTÃO 6 · CENÁRIO

O encapsulamento está corretamente aplicado neste sistema bancário?

java

```java
public class Conta {
    private double saldo;
    private double limiteChequeEspecial;

    public Conta(double saldo, double limiteChequeEspecial) {
        this.saldo = saldo;
        this.limiteChequeEspecial = limiteChequeEspecial;
    }
    public double getSaldo() { return saldo; }
    public void setSaldo(double saldo) { this.saldo = saldo; }
    public double getLimiteChequeEspecial() { return limiteChequeEspecial; }
}

public class ServicoTransferencia {
    public boolean transferir(Conta origem, Conta destino, double valor) {
        if (origem.getSaldo() + origem.getLimiteChequeEspecial() >= valor) {
            origem.setSaldo(origem.getSaldo() - valor);
            destino.setSaldo(destino.getSaldo() + valor);
            return true;
        }
        return false;
    }
}
```

Q6

QUESTÃO 6 · ALTERNATIVAS

Qual decisão aplica corretamente o encapsulamento?

A O encapsulamento já está correto: os atributos são private, impedindo acesso direto. Getters e setters transferem adequadamente a lógica de alteração de estado ao serviço externo, favorecendo a coesão.

B A visibilidade deve ser flexibilizada para protected, permitindo que ServicoTransferencia acesse as propriedades sem intermediários, centralizando as regras de negócio no mesmo pacote.

C O estado interno deveria ser imutável, evitando efeitos colaterais. O encapsulamento ideal seria remover os setters e exigir que o serviço recrie instâncias de Conta com os valores consolidados a cada transação.

D A classe deve concentrar a responsabilidade sobre suas invariantes: remover o setSaldo e criar operações como sacar(valor) e depositar(valor) dentro de Conta. Isso protege o estado interno e reduz o acoplamento com o serviço.

Q6

QUESTÃO 6 · GABARITO COMENTADO

Resposta Correta: Alternativa D

✓ **Por que a alternativa D está correta**

A solução correta remove o setSaldo público (que permite qualquer alteração arbitrária) e cria os métodos sacar(valor) e depositar(valor) DENTRO da própria classe Conta. Assim, a classe passa a proteger sua invariante (saldo + limite do cheque especial), e o ServicoTransferencia deixa de conhecer os detalhes internos de cálculo — reduzindo o acoplamento entre as classes.

POR QUE AS DEMAIS ALTERNATIVAS ERRAM

A Confunde "atributos private + getters/setters públicos" com bom encapsulamento. Expor setSaldo livremente permite manipulação do saldo sem validação — sintoma clássico do Modelo de Domínio Anêmico.

B Sugere ampliar a visibilidade para protected, o que só piora o problema: amplia o acesso ao estado interno para todo o pacote e subclasses, em vez de concentrá-lo na própria classe.

C Confunde encapsulamento com imutabilidade estrita. Uma entidade com ciclo de vida dinâmico (Conta) pode mudar de estado, desde que validado internamente; recriar a conta a cada transação é semanticamente inadequado.

Q6

QUESTÃO 6 · CONCEITO CONSOLIDADO

Encapsulamento Real — "Tell, Don't Ask"

- **Encapsulamento vai além de "private":** seu objetivo é garantir que o OBJETO tenha controle exclusivo sobre seu próprio estado interno e proteja suas invariantes (regras que sempre devem valer).

- **Modelo de Domínio Anêmico (Martin Fowler):** anti-padrão em que classes de domínio viram meros contêineres de dados (getters/setters públicos) e toda a lógica de negócio migra para classes de serviço externas, gerando alto acoplamento.

- **Princípio "Tell, Don't Ask":** em vez de perguntar o estado de um objeto e decidir por fora o que fazer, deve-se mandar o próprio objeto executar a operação (ex.: conta.sacar(valor) em vez de conta.setSaldo(...)).

- **Encapsulamento ≠ imutabilidade:** entidades com ciclo de vida dinâmico (como uma Conta) podem mudar de estado — desde que a mudança seja validada e executada pela própria classe.

PRINCÍPIO-CHAVE Tell, Don't Ask: não pergunte o estado do objeto — mande o próprio objeto agir sobre ele

FECHAMENTO

Os 6 pilares em uma linha cada

Q1 **Polimorfismo** — Atributos são estáticos, métodos sobrescritos são dinâmicos — nunca confunda os dois.

Q2 **Herança x Composição** — Herde só quando existir um verdadeiro "é-um"; para reaproveitar, componha.

Q3 **UML** — Composição = losango preenchido no todo; leia a multiplicidade do lado oposto.

Q4 **Strategy (GoF)** — Condicionais crescentes são sinal de que os algoritmos deveriam ser objetos.

Q5 **Abstrata x Interface** — Estado compartilhado pede classe abstrata; contrato transversal pede interface.

Q6 **Encapsulamento** — Tell, Don't Ask: o objeto decide como seu próprio estado muda.

REFERÊNCIAS

Bloch, J. Effective Java (3ª ed.), Addison-Wesley, 2018 · Gamma, Helm, Johnson & Vlissides. Design Patterns, Addison-Wesley, 1994 · Martin, R. C. Agile Software Development, Prentice Hall, 2002 · OMG. UML Specification v2.5.1 · Fowler, M. Anemic Domain Model (martinfowler.com) · Sommerville, I. Engenharia de Software (10ª ed.), Pearson, 2019 · Oracle. The Java Tutorials.